

Take-Home Assignment: Python Developer Intern

Role: Python Developer Intern

Time allowed: 3 days from receipt

Submission: Google Drive folder link shared to maria@sustainablelivinglab.org

Note: This is a fictional scenario designed to assess your skills. The system described below does not exist, is not in use, and will not be used by SL2 or any other organisation. We are evaluating how you think and build, not collecting a deliverable.

Before You Start

Don't spend more than 2–3 days on this. That is the intended scope. We only want to see what you can build in that window, not a polished product you've spent a week on.

Send us your submission even if it's incomplete. A partial API that works is far more useful to us than nothing. What's the worst that can happen? You lose 2–3 days and we give you honest feedback. Don't give up because something broke – document it and submit anyway.

We are not just looking for Python developers. We are looking for AI-first builders. This means we expect you to already be using tools like Cursor, Copilot, or similar in your daily workflow and know how to get the most out of them. How you use AI to move faster is fair game – what we're evaluating is the quality of your judgment and the output, not whether you typed every character yourself.

This is a remote-first role. Clear written communication matters as much as the code. Your README is part of the assessment – it should be the kind of document you'd send to a teammate who just joined, not a formality you rushed at the end.

Scenario

You are building and deploying the backend API for a prototype attendance management system for a fictional state-level skilling programme called **SkillBridge**. The system has five user roles, each with distinct permissions:

- **Student:** self-marks attendance for sessions they are enrolled in
- **Trainer:** creates sessions, manages batches, generates student invite links
- **Institution:** oversees trainers and batches under their institution
- **Programme Manager:** views attendance data across all institutions in a region
- **Monitoring Officer:** read-only access across the entire programme; cannot create, edit, or delete anything

Your task is to design, implement, and deploy the full REST API for this system in Python, with role-based access control enforced on every protected endpoint and a JWT-based access layer for the Monitoring Officer role.

Recommended Stack

You are free to use what you know. If you're starting fresh, these all have free tiers:

Layer	Recommended Option
Framework	FastAPI (preferred) or Flask
Database	Neon (PostgreSQL)
Deployment	Railway / Render / Fly.io
Auth tokens	PyJWT or python-jose

If you use a different service or library for any layer, document your choice in the README.

Tasks

Task 1: Data Model and Core API

Design and implement a REST API with the following schema and endpoints.

Entities:

- users: id, name, email, hashed_password, role (student / trainer / institution / programme_manager / monitoring_officer), institution_id (nullable), created_at
- batches: id, name, institution_id, created_at
- batch_trainers: batch_id, trainer_id (many-to-many; multiple trainers can be assigned to the same batch)
- batch_students: batch_id, student_id
- batch_invites: id, batch_id, token, created_by, expires_at, used (boolean)
- sessions: id, batch_id, trainer_id, title, date, start_time, end_time, created_at
- attendance: id, session_id, student_id, status (present / absent / late), marked_at

Endpoints:

Method	Path	Who
POST	/auth/signup	All roles
POST	/auth/login	All roles (returns JWT)
POST	/batches	Trainer / Institution
POST	/batches/{id}/invite	Trainer: generates a batch invite link/token
POST	/batches/join	Student: uses invite token to join batch
POST	/sessions	Trainer: title, date, start_time, end_time, batch_id
POST	/attendance/mark	Student: marks own attendance for an active session

Method	Path	Who
GET	/sessions/{id}/attendance	Trainer: full attendance list for a session
GET	/batches/{id}/summary	Institution: attendance summary for a batch
GET	/institutions/{id}/summary	Programme Manager: summary across all batches in an institution
GET	/programme/summary	Programme Manager: programme-wide summary
GET	/monitoring/attendance	Monitoring Officer: read-only, JWT-secured (see Task 2)

Every protected endpoint must extract the caller's role from their JWT and return 403 if the role is not permitted. Do not rely on the frontend to enforce access control.

Include a seed script that creates at least: 2 institutions, 4 trainers, 15 students, 3 batches, and 8 sessions with attendance records, so the summary endpoints return meaningful data.

Task 2: JWT Authentication and Role-Based Access Control

Implement authentication across the full API.

Signup and login:

- POST /auth/signup: accepts name, email, password, and role; stores a hashed password (use bcrypt or passlib); returns a JWT
- POST /auth/login: validates credentials, returns a signed JWT containing at minimum: user_id, role, iat, exp
- Token expiry: 24 hours for standard roles

Monitoring Officer – additional access control:

The Monitoring Officer role has an extra layer of access control on top of standard login:

- POST /auth/monitoring-token: accepts { "key": "<api_key>" } alongside a valid Monitoring Officer JWT; returns a short-lived read-only token (1 hour expiry) scoped only to monitoring endpoints
- GET /monitoring/attendance must validate this scoped token, not the standard login token
- Reject with 401 if the token is missing, expired, or issued for a different role
- Hardcode one valid API key in your .env file for testing

In your README, document:

- The full JWT payload structure for each token type
- How you would rotate or revoke tokens in a real deployment
- One security issue in your current implementation and how you'd fix it with more time

Task 3: Validation, Error Handling, and Tests

Validation and error handling:

- All POST endpoints must validate required fields and return 422 with a descriptive error body on failure, not a raw database exception
- Foreign key violations (e.g. referencing a non-existent batch_id or session_id) must return 404, not 500
- A student attempting to mark attendance for a session they are not enrolled in must receive 403
- The /monitoring/attendance endpoint must return 405 Method Not Allowed for any non-GET request

Tests: write at least five pytest tests covering:

1. Successful student signup and login, asserting a valid JWT is returned
2. A trainer creating a session with all required fields
3. A student successfully marking their own attendance
4. A POST to /monitoring/attendance returning 405
5. A request to a protected endpoint with no token returning 401

Tests must be runnable with pytest from the command line. Do not mock the database for all tests – at least two tests should hit a real (test) database.

Task 4: Deployment

Deploy the API so it is accessible via a public URL when you submit.

- API deployed to Railway, Render, or Fly.io (or equivalent)
- Database hosted on Neon or another managed PostgreSQL service
- Environment variables managed via the platform's secrets/config – no credentials committed to the repo

In your README, include:

- The live base URL of your deployed API
- A working curl command to hit /auth/login against the live deployment
- At least one seeded test account per role (email + password) that we can use to verify each access level

If deployment fails or is incomplete, document exactly what you attempted and where it broke. A partial deployment with honest documentation is preferable to a claimed full deployment that returns errors.

Task 5: README

Your README must cover:

1. Live API base URL and any relevant deployment notes
2. Local setup instructions from scratch (assume Python and pip are installed, nothing else)
3. Test accounts for all five roles (email and password)
4. Sample curl commands for every endpoint, including how to obtain and use both token types
5. Schema decisions: briefly explain the key modelling choices (especially batch_trainers, batch_invites, and the dual-token approach for Monitoring Officer)
6. What is fully working, what is partially done, and what you skipped

7. One thing you'd do differently with more time

What We Are Not Expecting

We are not expecting a production-grade system or perfect test coverage. We are expecting a deployed API where the core flows work, access control is enforced server-side, and the JWT implementation is correct.

A submission with Tasks 1–3 working cleanly on a live deployment, five passing tests, and an honest README beats one that claims all five tasks but has broken endpoints or tests that only pass locally.

We can tell when AI wrote it. Perfectly structured code that has never been run, tests that mock away everything real, a README describing a system different from what's in the repo – all of these get rejected. Show us something that was built, deployed, broken, and fixed.

Submission Format

```
/submission
  CONTACT.txt
  /src
  /tests
  README.md
  requirements.txt
  .env.example
```

CONTACT.txt must include your full name, email, phone, GitHub profile link, and one sentence on which part of the build you found most challenging.

Do not submit a Jupyter notebook as the main deliverable. The API must be runnable from the command line.

After You Submit

If shortlisted, we'll schedule a 30–45 minute call. We will hit your live API endpoints directly, log in as different roles, and ask you to walk through your design decisions. We may ask you to make a small change and redeploy. If you built it, this conversation will be easy.